

This is a copy of a shared ChatGPT conversation

Report conversation

Please search all concentration indices currently put forward. I only have to look into my methodology QMD. I have listed 3. I want to see if we have more

I'll check your methodology draft first to see exactly which three indices you already use, then cross-check the health inequality literature for other concentration-index variants that are worth mentioning or excluding.

I found your current QMD section: it already lists the standard CI, the generalised/absolute CI, and the corrected bounded-outcome index, then uses those three in the split rules. One immediate point: in your current tree split setting, CI_c is only a positive rescaling of CI_g when the outcome bounds are fixed, so it will not create a different split ranking.

Thought for 2m 7s >

Your QMD currently has the **three core indices**:

1. **Standard / relative concentration index:** CI
2. **Generalised / absolute concentration index:** CI_g
3. **Erreygers corrected index for bounded outcomes:** CI_c

That is a solid core set. However, the broader concentration-index literature has more variants. The main ones worth knowing are below.

Index	Common notation	Main idea	Worth adding to your thesis?
Standard / relative concentration index	CI , C , RCI	Relative socioeconomic-related inequality. Scale invariant for ratio-scale outcomes.	Already included.
Generalised concentration index	CI_g , GC , GCI	Absolute inequality: $CI_g = \mu \times CI$.	Already included.
Erreygers index	CI_c , E	Bounded-outcome version based on the generalised CI; for binary outcomes, $E = 4\mu CI = 4CI_g$.	Already included.

Index	Common notation	Main idea	Worth adding to your thesis?
Wagstaff normalised index	CI_W, W	Normalises the standard CI for bounded/binary variables so the range is $[-1, 1]$.	Yes, consider adding as a sensitivity index.
Modified concentration index	MC	For cardinal variables where the zero/lower point may be arbitrary.	Probably not useful for your binary U5MR setting.
Extended concentration index	$EC(v)$ or $C(v)$	Adds an inequality-aversion parameter v ; larger v gives more weight to poorer-ranked individuals.	Optional; useful only if you want a normative sensitivity analysis.
Generalised extended concentration index	$GEC(v)$	Absolute/generalised version of the extended CI.	Optional; probably too much for the main thesis.
Symmetric concentration index	$SC(\beta)$	Alternative rank-weighting scheme; can weight the middle or extremes of the SES distribution differently.	Optional; more theoretical.
Generalised symmetric concentration index	$GSC(\beta)$	Absolute/generalised version of the symmetric CI.	Optional; likely unnecessary.
Categorical/ordinal health inequality indices	varies	Developed for ordinal/categorical health variables where ordinary CI values can depend on arbitrary category scores.	Not needed for binary mortality unless you later analyse ordinal outcomes.

The practical software literature supports this classification. The R `rineq` package lists four main implemented types: relative CI, generalised CI, Erreygers correction, and Wagstaff bounded-variable index. R Project Search The Stata `conindex` paper includes an even broader family: standard, generalised, modified, Wagstaff, Erreygers, extended, and symmetric concentration indices. Sage Journals

For your thesis, the most important missing one is the **Wagstaff normalised concentration index** (CI_W). It is commonly discussed alongside Erreygers for bounded or binary health outcomes. Recent reviews still treat Wagstaff and Erreygers as the two main alternatives

for bounded health variables, but they also warn that these are not simple “corrections”; they encode different assumptions about inequality. Springer

My recommendation: keep your current three as the main framework, but add a short paragraph saying that the wider family also includes **Wagstaff’s normalised index, modified CI, extended CI, and symmetric CI**. Then state that, for this thesis, only CI , CI_g , and CI_c are used in the main splitting comparison, with CI_w considered as an additional sensitivity option.

A clean sentence for your QMD would be:

Other concentration-index variants have also been proposed, including Wagstaff’s normalised index for bounded variables, the modified concentration index for cardinal variables, and extended or symmetric concentration indices that change the rank-weighting structure. These measures are useful in specific settings, but the main comparison in this thesis focuses on the standard concentration index, the generalised concentration index, and the Erreygers corrected index, because these directly represent the relative, absolute, and bounded-outcome perspectives most relevant to binary under-five mortality.

One technical point: in your current split-gain derivation, CI_c will not produce a new split ranking relative to CI_g when the outcome bounds are fixed, because it is just a positive rescaling of CI_g . That is not a problem, but it is good to state clearly.

 Sources

what about the ones proposed egryers ?

Thought for 45s >

Yes — for **Erreygers**, the key one is:

1. Erreygers corrected concentration index

This is usually written as:

$$E = \frac{4}{b-a} GC$$

where:

$$GC = \mu C$$

So equivalently:

$$E = \frac{4\mu}{b-a} C$$

For a binary outcome such as under-five mortality, where $y \in [0, 1]$, this becomes:

$$E = 4\mu C$$

So in your thesis notation, your CI_c is essentially the **Erreygers index**.

Erreygers proposed it because the standard concentration index has problems for **bounded variables**, especially binary health outcomes. His correction was designed to satisfy four requirements: **transfer**, **level independence**, **cardinal invariance**, and the **mirror property**. The original paper also notes that the corrected index can be decomposed and generalised. IDEAS/RePEc

2. What is special about Erreygers compared with Wagstaff?

The difference is mostly conceptual.

Index	Logic	Interpretation
Wagstaff normalised CI	Relative bounded-variable correction	Adjusts the standard CI to fit binary bounds
Erreygers index	Absolute bounded-variable correction	Based on the generalised CI, so it reflects absolute inequality

For your work, Erreygers is very relevant because **under-five mortality is binary/bounded**, and because your tree impurity is based on inequality inside nodes. But note the important point:

$$E = 4CI_g$$

when $y \in [0, 1]$. So if your split rule uses CI_g and CI_c , both will give the **same split ordering**, because Erreygers is only a positive rescaling of the generalised concentration index. It may change the numerical scale, but not which split is selected.

3. Other Erreygers-related indices

There are a few related contributions, but they are not all necessary for your thesis.

Erreygers-related measure	What it does	Use in your thesis?
Erreygers corrected CI	Corrects CI for bounded variables	Yes — already your CI_c

Erreygers-related measure	What it does	Use in your thesis?
Generalised Erreygers-type index	Generalised/decomposable version of the corrected CI	Mention briefly if needed
Symmetric index	Alternative to the extended CI with a symmetry property	Probably not needed
Generalised symmetric index	Bounded/mirror-consistent version of the symmetric index	Probably not needed
Level-dependent bivariate indices	Uses SES levels, not only SES ranks	Too far from your current rank-based CI framework

Erreygers, Clarke and Van Ourti later discussed the **symmetric index** and its generalised version, contrasting it with the extended concentration index. Their point was that different rank-weighting choices change how much attention is given to different parts of the socioeconomic distribution. IDEAS/RePEc

How I would handle this in your methodology

You should not list many Erreygers indices as if they are all central. I would say:

The Erreygers corrected concentration index is included as the main bounded-outcome correction. For a binary outcome, it is given by $E = 4\mu C = 4CI_g$, where C is the standard concentration index, μ is the outcome mean, and CI_g is the generalised concentration index. This means that, in the present tree-splitting framework, the Erreygers index provides the bounded-variable interpretation of absolute inequality but does not alter split rankings relative to CI_g , since it is a positive rescaling when the outcome bounds are fixed.

So the answer is: **you already have the main Erreygers index**. The only additional one I would seriously add to your comparison is **Wagstaff's normalised index**, because it is the natural counterpart debated against Erreygers for binary outcomes.

Sources

this the one my lecturer wanted the L indices
Level-dependent bivariate indices

Yes, that sounds like the specific family your lecturer meant. I'll verify the exact terminology and how the "L" versions differ from the rank-based concentration index, then translate it into your methodology setting.

Thought for 1m 16s >

Exactly. The missing family is probably **Erreygers and Kessels' level-dependent bivariate indices**, often written as the ***L* index** family.

They are not just another "corrected concentration index." They are a different approach because they use **SES levels**, not only **SES ranks**.

1. Rank-dependent vs level-dependent

Your current concentration indices are **rank-dependent**:

$$CI, CI_g, CI_c$$

They rank people from poorest to richest, then measure whether the health outcome is concentrated among poorer or richer people.

The ***L* index** instead uses the actual socioeconomic level, for example income or wealth level:

$$L = \frac{1}{n} \sum_{i=1}^n \left(\frac{s_i - \mu_s}{\mu_s} \right) h_i$$

where:

- s_i = socioeconomic level of individual i , for example income
- μ_s = mean socioeconomic level
- h_i = health outcome

So the weight is:

$$w_i^L = \frac{s_i - \mu_s}{\mu_s}$$

This contrasts with the usual rank-dependent concentration-index weight, which is based on rank position rather than the actual income/wealth distance. The Economic Record paper describes this distinction directly: rank-dependent indices rely only on income ranks, while level-dependent indices use income levels; the basic ***L* index** uses a linear function of income levels. Research Explorer

2. Why your lecturer may care about this

The key motivation is that the ordinary concentration index ignores the **distance between socioeconomic levels**.

Example:

- Person A is poorer than Person B.
- The concentration index mainly cares that A ranks below B.
- The L index also cares **how much poorer** A is than B.

So if the richest households are extremely far from the mean wealth level, the L index gives them larger positive weights. This is why it is called **level-dependent**.

3. Bounded version for binary outcomes

For bounded health variables, Erreygers-type bounded versions can also be applied. The general bounded transformation is:

$$I^{bou} = \frac{A}{h_{\max} - h_{\min}} I$$

For the rank-dependent version, $A = 4$, which gives the Erreygers corrected index. For the level-dependent L version, the cited paper says the natural choice is $A = 1$. Research Explorer

For your binary under-five mortality outcome:

$$h_i \in [0, 1]$$

so:

$$h_{\max} - h_{\min} = 1$$

and therefore:

$$L^{bou} = L$$

So for your setting, the bounded L index is numerically the same as the basic L index, assuming the outcome is coded 0/1.

4. The important warning for your thesis

This is the main problem: your socioeconomic variable is likely the **DHS wealth index score**.

The L index works best when the SES variable has meaningful **levels**, such as income, consumption, or expenditure. But the DHS wealth index is usually a **PCA-based relative score**. It can be negative, centred around zero, and its scale is not naturally interpretable.

That matters because:

$$L = \frac{1}{n} \sum_i \left(\frac{s_i - \mu_s}{\mu_s} \right) h_i$$

requires μ_s to be meaningful. If the DHS wealth score has mean near zero, this index becomes unstable or meaningless. Also, shifting the wealth score by a constant would change the L index, whereas rank-based concentration indices would not be affected by monotone transformations.

So I would include the L index in the methodology, but probably as:

a theoretically relevant alternative, not necessarily part of the main empirical split rule unless a meaningful level-based SES variable is available.

5. How to add it to your QMD

You can add a subsection like this:

Markdown

```
#### Level-dependent bivariate index
```

In addition to rank-dependent concentration indices, Erreygers and Kessels proposed level-dep

```
$$
L = \frac{1}{n} \sum_{i=1}^n
\left( \frac{s_i - \mu_s}{\mu_s} \right) y_i,
$$
```

where s_i denotes the socioeconomic level, μ_s its mean, and y_i the health outcome.

This index differs from the concentration index because it uses socioeconomic levels rather t

However, this measure requires the socioeconomic variable to have a meaningful level scale. 1

6. R function for the L index

R

```
l_index <- function(y, ses, weights = NULL, bounded = FALSE,
                    h_min = 0, h_max = 1) {
  if (is.null(weights)) {
    weights <- rep(1, length(y))
  }

  p <- weights / sum(weights)
  mu_s <- sum(p * ses)
```

```

if (abs(mu_s) < 1e-8) {
  stop("Mean SES is close to zero. The L index is not stable for this SES scale.")
}

L <- sum(p * ((ses - mu_s) / mu_s) * y)

if (bounded) {
  L <- L / (h_max - h_min)
}

return(L)
}

```

For your list of indices, update it to:

1. CI : standard/rank-dependent relative concentration index
2. CI_g : generalised concentration index
3. CI_c or E : Erreygers corrected index
4. W : Wagstaff normalised index
5. L : level-dependent bivariate index
6. L^{bou} : bounded level-dependent index

The lecturer's "L indices" almost certainly refer to items 5 and 6.

Sources

given my scripts write and instructions to llm telling it to implement the L indice such that it fits right in with my current framework

this is the github folder create links using the example link below

https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/main/R/rank_wt.R

You can access the other code as below

Show more 

I have enough of the package structure now. The main change is small but must be placed carefully: add the L index inside `ci_factory()`, then let the existing `gain`, `tree`, `forest`, and `summary` functions inherit it through the existing `ci_fun` interface.

Thought for 2m 10s >

Below is a copy-paste instruction prompt you can give to another LLM or coding agent. It is written to fit your current `ineqTrees` structure.

Your current package already centralises split scoring through `ci_factory()`, which takes a two-column response matrix and a case-weight vector, then returns a non-negative index value. The first column is currently treated as the socioeconomic variable and ranked internally for `CI`, `CIg`, and `CIc`; the split gain then compares the parent index with the weighted child-node indices. [GitHub](#) The rest of the tree and forest code already calls `ci_factory(type)` and passes the resulting `ci_fun` into the split search, so the cleanest implementation is to add `"L"` to this factory and propagate it through the existing `type` arguments. [GitHub](#) **+1**

You are working on the R package ``ineqTrees``.

Goal:

Implement the Erreygers-Kessels level-dependent bivariate index, here called ``"L"``, so

Important literature context:

The package currently implements rank-dependent concentration-index variants. The new

- <https://www.mdpi.com/1660-4601/14/7/673>
- <https://ideas.repec.org/a/gam/jijerp/v14y2017i7p673-d102411.html>

Relevant package files:

- ``rank_wt.R``: <https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/main/R/r>
- ``fractional_rank.R``: <https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/>
- ``best_numeric_split.R``: <https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/>
- ``ci_splitfun.R``: <https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/main>
- ``ci_tree_control.R``: <https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/>
- ``shap_conc_decomp.R``: <https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/>
- ``NAMESPACE``: <https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/main/NAM>
- ``DESCRIPTION``: <https://raw.githubusercontent.com/m-mburu/ineqTrees/refs/heads/main/D>

Current architecture to preserve:

1. ``ci_factory(type)`` is the central scoring factory.
2. ``weighted_ci_gain()`` takes a ``ci_fun`` and computes:
parent criterion - weighted average child criterion.
3. ``best_numeric_split()``, ``best_factor_split()``, and ``best_global_ci_split()`` should
4. ``ci_splitfun()``, ``ctree_ci()``, and ``cf_ci()`` currently accept ``type = c("CI", "CIg"`
5. Do not rewrite the tree builder. Add the new index through the existing ``ci_fun`` in
6. Preserve backwards compatibility: existing calls using ``"CI"``, ``"CIg"``, and ``"CIc"``

Definition to implement:

For observations $\backslash(i = 1, \ldots, n\backslash)$, let:

- $\backslash(s_i\backslash)$ be the socioeconomic level variable.
- $\backslash(y_i\backslash)$ be the health outcome.

- w_i be the case weight.
- $p_i = w_i / \sum_i w_i$.
- $\mu_s = \sum_i p_i s_i$.

The level-dependent bivariate index should be:

$$L = \sum_i p_i \left(\frac{s_i - \mu_s}{\mu_s} \right) y_i.$$

For split scoring, return:

$$|L|.$$

Important:

For `"L"`, do NOT convert the socioeconomic variable to fractional ranks. Use the obse

Implementation details:

1. Modify `ci_factory()` in `rank_wt.R`.

Current signature is conceptually:

```

```r
ci_factory <- function(type = c("CI", "CIg", "CIc")) {
 type <- match.arg(type)
 function(y, wt) {
 ...
 }
}

```

Change to:

```

R

ci_factory <- function(type = c("CI", "CIg", "CIc", "L")) {
 type <- match.arg(type)
 function(y, wt) {
 ...
 }
}

```

Inside the returned function:

- Keep the existing validation:
  - `y` must be a two-column matrix.
  - `length(wt) == nrow(y)`.
  - Drop incomplete rows.

- Drop missing or non-positive weights.
- Return 0 when fewer than two valid observations remain.
- Store the original columns before ranking:

R

```
ses <- y[, 1]
outcome <- y[, 2]
```

- Add this branch before the rank-based CI calculations:

R

```
if (type == "L") {
 mu_s <- stats::weighted.mean(ses, wt)

 if (isTRUE(all.equal(mu_s, 0))) {
 return(0)
 }

 p <- wt / sum(wt)
 L <- sum(p * ((ses - mu_s) / mu_s) * outcome)

 if (!is.finite(L)) {
 return(0)
 }

 return(abs(L))
}
```

Then keep the existing rank-dependent logic for `CI`, `CIg`, and `CIc`.

2. Update roxygen documentation in `rank_wt.R`.

Change the `@param type` description from:

R

```
One of "CI", "CIg", or "CIc"
```

to:

R

```
One of "CI", "CIg", "CIc", or "L".
"L" selects the Erreygers--Kessels level-dependent bivariate index.
For "L", the first response column is used as a socioeconomic level variable, not converted
```

Also update the details section:

- Explain that  $CI$ ,  $CIg$ , and  $CIc$  are rank-dependent.
- Explain that  $L$  is level-dependent and uses:

R

```
sum(p * ((ses - mu_s) / mu_s) * outcome)
```

3. Update all `type` arguments in tree and forest functions.

In `ci_splitfun.R`, update every user-facing or internal function that currently has:

R

```
type = c("CI", "CIg", "CIc")
```

to:

R

```
type = c("CI", "CIg", "CIc", "L")
```

This includes at least:

- `ci_splitfun()`
- `ctree_ci()`
- `cf_ci()`

Also update roxygen docs for:

- `ci_splitfun()`
- `ctree_ci()`
- `cf_ci()`

Use language like:

R

```
@param type One of ``CI``, ``CIg``, ``CIc``, or ``L`` selecting the inequality index used for
```



4. Do not change `weighted_ci_gain()`.

It already accepts any `ci_fun` with signature:

R

```
function(y, wt)
```

The "L" index should work automatically because the gain formula is generic:

R

```
ci_parent - ((wl / wtot) * ci_left + (wr / wtot) * ci_right)
```

5. Do not change `best_numeric_split()`, `best_factor_split()`, or `best_global_ci_split()` unless needed for tests.

These functions should remain index-agnostic. They should keep using `ci_fun`.

6. Update summaries.

`ci_tree_terminal_summary()` and `ci_forest_summary()` already call `ci_factory(object$type)` or read stored node information. They should work after "L" is added. Only update documentation or labels if needed. Do not rename the `ci` column unless you also update all downstream code. The column can remain `ci`, but document that it stores the selected inequality criterion, including "L".

7. Add tests.

Create testthat tests for `ci_factory("L")`.

Minimum tests:

R

```
test_that("ci_factory L computes level-dependent bivariate index", {
 y <- cbind(
 ses = c(1, 2, 3),
 outcome = c(0, 1, 1)
)
 wt <- c(1, 1, 1)

 l_fun <- ci_factory("L")

 # mu_s = 2
 # weights: (-0.5, 0, 0.5)
 # L = mean(c(0, 0, 0.5)) = 1/6
 expect_equal(l_fun(y, wt), 1 / 6)
})
```

Add a neutrality test:

R

```
test_that("L index is zero when outcome is constant", {
 y <- cbind(
 ses = c(1, 2, 3, 4),
 outcome = c(1, 1, 1, 1)
)
```

```

)
wt <- rep(1, 4)

expect_equal(ci_factory("L")(y, wt), 0)
})

```

Add a unit-consistency test:

```

R

test_that("L index is invariant to positive rescaling of SES", {
 y1 <- cbind(
 ses = c(1, 2, 3, 4),
 outcome = c(0, 1, 0, 1)
)
 y2 <- cbind(
 ses = 10 * y1[, "ses"],
 outcome = y1[, "outcome"]
)
 wt <- rep(1, 4)

 expect_equal(ci_factory("L")(y1, wt), ci_factory("L")(y2, wt))
})

```

Add an integration test for the split search:

```

R

test_that("ctree_ci accepts type L", {
 toy_data <- data.frame(
 ses = c(1, 2, 3, 4, 5, 6),
 outcome = c(1, 1, 0, 0, 0, 1),
 x = c(1, 1, 2, 2, 3, 3)
)

 fit <- ctree_ci(
 formula = cbind(ses, outcome) ~ x,
 data = toy_data,
 rank_name = "ses",
 outcome_name = "outcome",
 type = "L",
 control = ci_tree_control(minsplit = 1, minbucket = 1, maxdepth = 1)
)

 expect_true(inherits(fit, "ci_tree"))
})

```

Add a forest test:

R

```
test_that("cf_ci accepts type L", {
 toy_data <- data.frame(
 ses = c(1, 2, 3, 4, 5, 6),
 outcome = c(1, 1, 0, 0, 0, 1),
 x = c(1, 1, 2, 2, 3, 3)
)

 fit <- cf_ci(
 formula = cbind(ses, outcome) ~ x,
 data = toy_data,
 rank_name = "ses",
 outcome_name = "outcome",
 type = "L",
 ntree = 3,
 control = ci_tree_control(minsplit = 1, minbucket = 1, maxdepth = 1)
)

 expect_true(inherits(fit, "ci_forest"))
})
```

8. Optional but recommended: add L support to `shap_conc_decomp()` .

Current `shap_conc_decomp()` computes SHAP contributions to a rank-dependent concentration index using fractional ranks. For consistency, add:

R

```
type = c("CI", "L")
```

For `type = "CI"` , keep the current behaviour exactly.

For `type = "L"` :

- Interpret the existing `rank` argument as the socioeconomic level variable.
- Compute:

R

```
ses <- working_dt[["rank__"]]
mu_s <- mean(ses)
level_weight <- (ses - mu_s) / mu_s
```

- Then compute each SHAP contribution as:

R

```
D_k_SHAP <- mean(level_weight * phi_k)
```

- Compute total index as:

R

```
total_index <- mean(level_weight * prediction_vector)
```

- The baseline contribution should be zero because  $\text{mean}(\text{level\_weight}) = 0$ , apart from numerical error.
- Update diagnostics column name carefully. To avoid breaking existing code, keep `concentration_index` as the diagnostic column, but add `type`.

If you add weights to `shap_conc_decomp()`, do it as a separate optional argument and preserve old default behaviour when `weights = NULL`.

## 9. Update examples.

Add examples using "L" :

R

```
l_index <- ci_factory("L")
y <- cbind(
 ses = c(1, 2, 3, 4),
 outcome = c(0, 1, 0, 1)
)
l_index(y, rep(1, 4))
```

And:

R

```
fit_l <- ctree_ci(
 cbind(ses, outcome) ~ x1 + x2,
 data = df,
 rank_name = "ses",
 outcome_name = "outcome",
 type = "L"
)
```

## 10. Run package checks.

Run:

R

```
devtools::document()
devtools::test()
```

```
devtools::check()
```

Make sure:

- Existing tests for "CI" , "CIg" , and "CIc" still pass.
- The `NAMESPACE` updates automatically if roxygen changes require it.
- No downstream function assumes `type` has only three possible values.

Important methodological warning to include in documentation:

The "L" index requires the first response column to be a meaningful socioeconomic level variable, ideally income, consumption, expenditure, or another non-negative ratio-scale SES measure. It should not be used naively with a centred PCA wealth-index score whose mean may be close to zero or whose level scale has no direct interpretation. If using DHS wealth scores, consider transforming them only if a defensible level interpretation exists, or treat "L" as a sensitivity analysis rather than the main criterion.

```
For your thesis package, I would implement only `"L"` first**, not `"Lrel"` or extend::contentReference[oaicite:2]{index=2}
```



 Sources